

动态规划

zlx

动态规划

动态规划是指多阶段决策过程的优化问题，通常会将问题划分成更简单的子问题或是阶段来完成。

动态规划

动态规划是指多阶段决策过程的优化问题，通常会将问题划分成更简单的子问题或是阶段来完成。

使用动态规划解决问题时一般会选择一些特殊的情况作为“状态”，处理状态之间的转移来求解问题，同时需要保证状态和转移的完整性。

动态规划

动态规划是指多阶段决策过程的优化问题，通常会将问题划分成更简单的子问题或是阶段来完成。

使用动态规划解决问题时一般会选择一些特殊的情况作为“状态”，处理状态之间的转移来求解问题，同时需要保证状态和转移的完整性。

解决具体问题时还需要压缩无用状态，合并相似状态来优化算法效率。

状态设计

设计一组优秀的状态是动态规划中最adhoc的部分，常常是一题一法，千题千面。

状态设计

设计一组优秀的状态是动态规划中最adhoc的部分，常常是一题一法，千题千面。

在具体问题中，分析转化、类比经典问题、与其他算法相结合等方法都可以帮助我们更好的解决问题。但不能拘泥于单独的一种方法，从不同的角度审视问题，寻找切入点并设计状态，或许会有新的发现。

CF1842G Tenzing and Random Operations

[CF1842G Tenzing and Random Operations](#)

CF1842G Tenzing and Random Operations

观察到 m 比较大，考虑给问题一个新的组合意义，使得复杂度与 m 无关。

CF1842G Tenzing and Random Operations

观察到 m 比较大，考虑给问题一个新的组合意义，使得复杂度与 m 无关。

把每个位置的权值写成 $a_i + v + v + \dots + v$ 的形式，考虑每个位置选取的是哪一个操作，这样有效的操作数量就只有 $O(n)$ 个了。

CF1842G Tenzing and Random Operations

观察到 m 比较大，考虑给问题一个新的组合意义，使得复杂度与 m 无关。

把每个位置的权值写成 $a_i + v + v + \dots + v$ 的形式，考虑每个位置选取的是哪一个操作，这样有效的操作数量就只有 $O(n)$ 个了。

设 $f_{i,j}$ 表示前 i 个位置，已经选取了 j 个不同的操作的情况的期望权值。

CF1842G Tenzing and Random Operations

观察到 m 比较大，考虑给问题一个新的组合意义，使得复杂度与 m 无关。

把每个位置的权值写成 $a_i + v + v + \dots + v$ 的形式，考虑每个位置选取的是哪一个操作，这样有效的操作数量就只有 $O(n)$ 个了。

设 $f_{i,j}$ 表示前 i 个位置，已经选取了 j 个不同的操作的情况的期望权值。

转移分成当前位置，选取 a_i ，选取之前出现过的操作，选取之前未出现过的操作，这三种情况，分别转移即可。时间复杂度 $O(n^2)$ 。

判定转dp

判定转dp常见于需要统计所有“合法”情况信息的问题中。

判定转dp

判定转dp常见于需要统计所有“合法”情况信息的问题中。

具体过程为：先设计一个能判定合法性的算法，再将判定合法性需要用到的信息设计进dp中，从而设计出一组状态。

AGC022E Median Replace

AGC022E Median Replace

AGC022E Median Replace

先来考虑一个如何判定一个串是美丽的。

AGC022E Median Replace

先来考虑一个如何判定一个串是美丽的。
有 000 显然直接消，111 显然不会动，剩下的情况等价于删除一对 01。

AGC022E Median Replace

先来考虑一个如何判定一个串是美丽的。

有 000 显然直接消，111 显然不会动，剩下的情况等价于删除一对 01。

因为需要易于dp，我们考虑逐个插入字符并贪心执行上面的操作。

AGC022E Median Replace

先来考虑一个如何判定一个串是美丽的。

有 000 显然直接消，111 显然不会动，剩下的情况等价于删除一对 01。

因为需要易于dp，我们考虑逐个插入字符并贪心执行上面的操作。

具体的，可以维护一个栈，其结构为上面全是 0 下面全是 1，加入 1 时可以和前面的 0 相互消除，加入 0 时如果前面有多于 3 个 0 则消除两个。

AGC022E Median Replace

先来考虑一个如何判定一个串是美丽的。

有 000 显然直接消，111 显然不会动，剩下的情况等价于删除一对 01。

因为需要易于dp，我们考虑逐个插入字符并贪心执行上面的操作。

具体的，可以维护一个栈，其结构为上面全是 0 下面全是 1，加入 1 时可以和前面的 0 相互消除，加入 0 时如果前面有多于 3 个 0 则消除两个。

容易发现 0 最多有两个，且如果有两个 1 就必胜了，那么状态数非常少，直接dp即可。

uoj748. [UNR #6] 机器人表演

uoj748. [UNR #6] 机器人表演

uoj748. [UNR #6] 机器人表演

考虑能不能用 S 生成 T 。贪心的，从前往后扫 S ，尽量长的与 T 进行匹配。但是这样可能会因为前面匹配太多而导致剩下的串不是合法的括号序列。

uoj748. [UNR #6] 机器人表演

考虑能不能用 S 生成 T 。贪心的，从前往后扫 S ，尽量长的与 T 进行匹配。但是这样可能会因为前面匹配太多而导致剩下的串不是合法的括号序列。

尝试反悔，等价于需要让 S 已匹配部分的 “(” 减少一个，那么一直回退到最近的合法位置即可，具体位置只与 S 有关，可以预处理。

uoj748. [UNR #6] 机器人表演

考虑能不能用 S 生成 T 。贪心的，从前往后扫 S ，尽量长的与 T 进行匹配。但是这样可能会因为前面匹配太多而导致剩下的串不是合法的括号序列。

尝试反悔，等价于需要让 S 已匹配部分的 “(” 减少一个，那么一直回退到最近的合法位置即可，具体位置只与 S 有关，可以预处理。

那么 dp 值需要记录 S, T 各自的匹配长度，以及 T 中未匹配部分的前缀和即可，复杂度 $O(n^3)$ 。

连续段dp

连续段dp是一类比较特殊的dp，可以用来处理如下问题：

对于所有长度为 n 的排列 p ，求：

$$\sum_p \left(\sum_{i>1} f(p_{i-1}, p_i) \right)$$

其中 $f(x, y)$ 一般为主要与 x, y 大小关系有关的函数。

连续段dp

连续段dp是一类比较特殊的dp，可以用来处理如下问题：

对于所有长度为 n 的排列 p ，求：

$$\sum_p \left(\sum_{i>1} f(p_{i-1}, p_i) \right)$$

其中 $f(x, y)$ 一般为主要与 x, y 大小关系有关的函数。

具体做法为：从小到大插入元素，并考虑它们在最终排列中的相邻情况，维护一些连续段，从而求解问题。

P9197 [JOI Open 2016] 摩天大楼

P9197 [JOI Open 2016] 摩天大楼

P9197 [JOI Open 2016] 摩天大楼

从小到大插入，维护当前的连续段数量。插入时考虑插入的位置，有挂在一个连续段两侧、合并两个连续段、单开一个连续段三种情况，分别计算权值即可，需要注意的是左右端点需要特殊维护，额外记录两个 01 状态即可。

P9197 [JOI Open 2016] 摩天大楼

从小到大插入，维护当前的连续段数量。插入时考虑插入的位置，有挂在一个连续段两侧、合并两个连续段、单开一个连续段三种情况，分别计算权值即可，需要注意的是左右端点需要特殊维护，额外记录两个 01 状态即可。

题目对总和的限制 L 不大，但直接加减并放入状态中，由于会有负数，因此复杂度过高。可以把和拆到每一对 a_i, a_{i+1} 之间，这样和这一维的一定非负。复杂度 $O(n^2L)$ 。

树上赋权

没有找到通用的算法名，但问题比较经典。

树上赋权

没有找到通用的算法名，但问题比较经典。
具体可以用来处理，用边限制两点大小关系，
且图类似树的合法赋权方案问题。

树上赋权

没有找到通用的算法名，但问题比较经典。

具体可以用来处理，用边限制两点大小关系，且图类似树的合法赋权方案问题。

举个例子：给一棵有根树随机一个排列作为点权，求每个点的点权都是它子树内点权的最大值的概率。

树上赋权

没有找到通用的算法名，但问题比较经典。

具体可以用来处理，用边限制两点大小关系，且图类似树的合法赋权方案问题。

举个例子：给一棵有根树随机一个排列作为点权，求每个点的点权都是它子树内点权的最大值的概率。

答案为 $\prod_{i=1}^n \frac{1}{size_i}$ ，其中点 x 满足条件的概率是 $\frac{1}{size_x}$ 。

树上赋权

更近一步的，在树的边方向任意的情况下，也可以用这种方法解决。

树上赋权

更近一步的，在树的边方向任意的情况下，也可以用这种方法解决。

具体的，任意取dfs树，对于树中的反向边，考虑容斥成 [没有这条边] - [有正向边]。就可以方便的使用dp维护了。

树上赋权

更近一步的，在树的边方向任意的情况下，也可以用这种方法解决。

具体的，任意取dfs树，对于树中的反向边，考虑容斥成 [没有这条边] - [有正向边]。就可以方便的使用dp维护了。

如果图在树的基础上又添加了几条边，也可以通过容斥的方法替换或拆除一些边，从而将结构变成一棵树。

AGC060C Large Heap

AGC060C Large Heap

AGC060C Large Heap

本题相当于在一棵树上多加了一条边，那么考虑 A, B 两点到根的路径上的所有点，如果能知道它们的大小关系，就可以重构这两条链，从而用上面的方法求出答案。

AGC060C Large Heap

本题相当于在一棵树上多加了一条边，那么考虑 A, B 两点到根的路径上的所有点，如果能知道它们的大小关系，就可以重构这两条链，从而用上面的方法求出答案。

那么考虑用dp处理整个过程，讨论最下面两个点的大小关系，类似归并的转移即可。

AGC060C Large Heap

本题相当于在一棵树上多加了一条边，那么考虑 A, B 两点到根的路径上的所有点，如果能知道它们的大小关系，就可以重构这两条链，从而用上面的方法求出答案。

那么考虑用dp处理整个过程，讨论最下面两个点的大小关系，类似归并的转移即可。

转移过程需要求逆元，暴力计算复杂度 $O(n^2 \log n)$ ，预处理即可做到 $O(n^2)$ 。

qoj8047. DFS Order 4

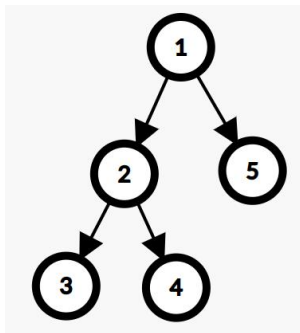
qoj8047. DFS Order 4

qoj8047. DFS Order 4

先考虑将一种合法的dfs序对应到唯一的一棵树上，从而值需要统计合法树的数量即可。

qoj8047. DFS Order 4

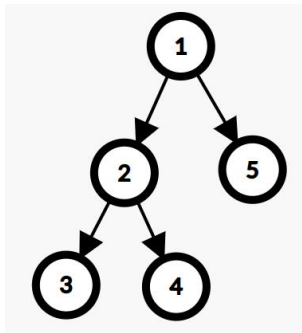
先考虑将一种合法的dfs序对应到唯一的一棵树上，从而值需要统计合法树的数量即可。



如图，对于 3,4 两个点，它们挂在 1 或 2 下面得到的dfs序是一样的，那么我们钦定这些点需要挂在节点“1”下面。

qoj8047. DFS Order 4

先考虑将一种合法的dfs序对应到唯一的一棵树上，从而值需要统计合法树的数量即可。



如图，对于 3,4 两个点，它们挂在 1 或 2 下面得到的dfs序是一样的，那么我们钦定这些点需要挂在节点“1”下面。

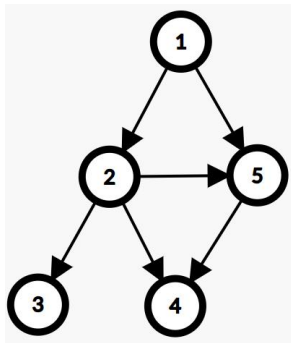
更进一步的，相当于要求一个点的最后一个儿子比它的兄弟大，那就是节点“4”的标号要比节点“5”大。

qoj8047. DFS Order 4

可以想到上述的条件已经可以使得dfs序与树有唯一对应关系了，考虑建出节点间大小关系图：

qoj8047. DFS Order 4

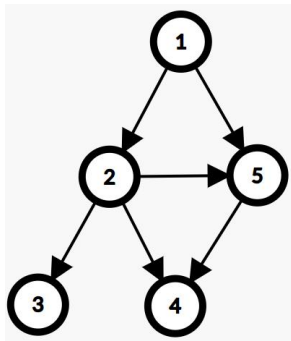
可以想到上述的条件已经可以使得dfs序与树有唯一对应关系了，考虑建出节点间大小关系图：



如图，有与祖先的边、与兄弟的边、最后一个儿子和兄弟的边这三类，但它们会形成环。

qoj8047. DFS Order 4

可以想到上述的条件已经可以使得dfs序与树有唯一对应关系了，考虑建出节点间大小关系图：



如图，有与祖先的边、与兄弟的边、最后一个儿子和兄弟的边这三类，但它们会形成环。

考虑到最后一个儿子和兄弟的边最特殊的，考虑容斥这条边。没有这条边是简单的，有一条反向边的话那么兄弟边就可以删除了。

qoj8047. DFS Order 4

考虑dp统计这些树的数量。

qoj8047. DFS Order 4

考虑dp统计这些树的数量。

由于大小关系上“子树”的大小并不等于树中的子树大小，还有自己及祖先的兄弟节点，因此需要额外设一维状态，表示这些点的大小总和。

qoj8047. DFS Order 4

考虑dp统计这些树的数量。

由于大小关系上“子树”的大小并不等于树中的子树大小，还有自己及祖先的兄弟节点，因此需要额外设一维状态，表示这些点的大小总和。

转移枚举第一个儿子的大小，考虑它的最后一个儿子和兄弟的连边情况即可。

qoj8047. DFS Order 4

考虑dp统计这些树的数量。

由于大小关系上“子树”的大小并不等于树中的子树大小，还有自己及祖先的兄弟节点，因此需要额外设一维状态，表示这些点的大小总和。

转移枚举第一个儿子的大小，考虑它的最后一个儿子和兄弟的连边情况即可。

复杂度 $O(n^3)$ 。

qoj1839. Joke

[qoj1839. Joke](#)

qoj1839. Joke

首先观察到 p 是不重要，以下默认 $p_i = i$ ，
且设 $b_i = a_{2,i}$ 。

qoj1839. Joke

首先观察到 p 是不重要，以下默认 $p_i = i$ ，
且设 $b_i = a_{2,i}$ 。

可以发现 a_1, a_2 两个序列本质上限定了一些
大小关系，我们用图中的边来刻画这些限制。

qoj1839. Joke

首先观察到 p 是不重要，以下默认 $p_i = i$ ，
且设 $b_i = a_{2,i}$ 。

可以发现 a_1, a_2 两个序列本质上限定了一些
大小关系，我们用图中的边来刻画这些限制。

考虑判定一组 s 是否可以被构造。 s 限定了
 $a_{1,i}, a_{2,i}$ 的大小关系，同样用边来表示。

qoj1839. Joke

首先观察到 p 是不重要，以下默认 $p_i = i$ ，
且设 $b_i = a_{2,i}$ 。

可以发现 a_1, a_2 两个序列本质上限定了一些
大小关系，我们用图中的边来刻画这些限制。

考虑判定一组 s 是否可以被构造。 s 限定了
 $a_{1,i}, a_{2,i}$ 的大小关系，同样用边来表示。

那么一组 s 可以被构造就转化为了对应的图
没有环，容易发现这等价于没有四元环。

qoj1839. Joke

回到序列上，判定条件即为不存在一对 (i, j) 使得 $i < j, b_i > b_j, s_i = 1, s_j = 0$ 。

qoj1839. Joke

回到序列上，判定条件即为不存在一对 (i, j) 使得 $i < j, b_i > b_j, s_i = 1, s_j = 0$ 。

将点 (i, b_i) 放到平面上，那么合法方案相当于存在一条折线，使得折线左上方的点满足 $s = 0$ ，折线右下方的点满足 $s = 1$ 。

qoj1839. Joke

回到序列上，判定条件即为不存在一对 (i, j) 使得 $i < j, b_i > b_j, s_i = 1, s_j = 0$ 。

将点 (i, b_i) 放到平面上，那么合法方案相当于存在一条折线，使得折线左上方的点满足 $s = 0$ ，折线右下方的点满足 $s = 1$ 。

可以发现本质不同的折线数量等于 b 序列的上升子序列数量。

qoj1839. Joke

回到序列上，判定条件即为不存在一对 (i, j) 使得 $i < j, b_i > b_j, s_i = 1, s_j = 0$ 。

将点 (i, b_i) 放到平面上，那么合法方案相当于存在一条折线，使得折线左上方的点满足 $s = 0$ ，折线右下方的点满足 $s = 1$ 。

可以发现本质不同的折线数量等于 b 序列的上升子序列数量。

简单设计dp即可做到 $O(n^4)$ 复杂度。

AGC020F Arcs on a Circle

AGC020F Arcs on a Circle

AGC020F Arcs on a Circle

题目中圆弧的起点位置是任意实数，显然不能直接枚举。

AGC020F Arcs on a Circle

题目中圆弧的起点位置是任意实数，显然不能直接枚举。

观察到我们只关心圆弧之间的相交关系，考虑类似于离散化，把绝对的值转化成相对的关系来处理。根据概率的性质，可以不考虑有两个小数部分相同的情况，且所有的离散化结果等概率。

AGC020F Arcs on a Circle

题目中圆弧的起点位置是任意实数，显然不能直接枚举。

观察到我们只关心圆弧之间的相交关系，考虑类似于离散化，把绝对的值转化成相对的关系来处理。根据概率的性质，可以不考虑有两个小数部分相同的情况，且所有的离散化结果等概率。

环较为复杂，考虑从最长的圆弧起点处将环断开，变成一条链。

AGC020F Arcs on a Circle

本题中的 n 不大，直接枚举离散化结果，使用动态规划计算对应的概率。

AGC020F Arcs on a Circle

本题中的 n 不大，直接枚举离散化结果，使用动态规划计算对应的概率。

具体的，设 $f_{i,j,s}$ 表示加入了左端点坐标小于等于 i 的线段，最远的右端点坐标为 j ，线段的使用情况为 S 的方案数。其中坐标可以用整数和一个离散化之后的小数表示。

AGC020F Arcs on a Circle

本题中的 n 不大，直接枚举离散化结果，使用动态规划计算对应的概率。

具体的，设 $f_{i,j,s}$ 表示加入了左端点坐标小于等于 i 的线段，最远的右端点坐标为 j ，线段的使用情况为 S 的方案数。其中坐标可以用整数和一个离散化之后的小数表示。

转移考虑是否放置线段即可。

AGC020F Arcs on a Circle

本题中的 n 不大，直接枚举离散化结果，使用动态规划计算对应的概率。

具体的，设 $f_{i,j,s}$ 表示加入了左端点坐标小于等于 i 的线段，最远的右端点坐标为 j ，线段的使用情况为 S 的方案数。其中坐标可以用整数和一个离散化之后的小数表示。

转移考虑是否放置线段即可。

复杂度 $O(n! 2^n (nc)^2)$ 。

优化转移

动态规划中，与状态对应的就是转移。

优化转移

动态规划中，与状态对应的就是转移。

相比于状态设计，转移以及其优化是更为套路的。删除无用转移、合并相似转移、性质/数据结构优化转移等都是常见技巧。

优化转移

动态规划中，与状态对应的就是转移。

相比于状态设计，转移以及其优化是更为套路的。删除无用转移、合并相似转移、性质/数据结构优化转移等都是常见技巧。

在分析优化前，把所有转移完整的列出通常可以让解题更顺利。

树上背包合并

树上背包合并是一类常见的动态规划，合理实现复杂度为 $O(nk)$ ，其中 k 是背包大小。

P4516 [JSOI2018] 潜入行动

P4516 [JSOI2018] 潜入行动

P4516 [JSOI2018] 潜入行动

经典的树上背包，可以直接设状态 $f_{x,i,0/1,0/1}$ 表示 x 子树内放置了 i 个监听器， x 上是否放置了监听器， x 是否已被监听。

P4516 [JSOI2018] 潜入行动

经典的树上背包，可以直接设状态 $f_{x,i,0/1,0/1}$ 表示 x 子树内放置了 i 个监听器， x 上是否放置了监听器， x 是否已被监听。

合并时， i 这一维直接加， $0/1$ 状态讨论一下合并即可，复杂度 $O(nk)$ 。

P10064 [SNOI2024] 公交线路

P10064 [SNOI2024] 公交线路

P10064 [SNOI2024] 公交线路

显然只需要考虑叶子节点间的距离。对于点 u ，设新图上与它直接相连的点集为 S_u 。题目等价于任意两点 u, v 都满足 S_u, S_v 的交非空。

P10064 [SNOI2024] 公交线路

显然只需要考虑叶子节点间的距离。对于点 u ，设新图上与它直接相连的点集为 S_u 。题目等价于任意两点 u, v 都满足 S_u, S_v 的交非空。

进一步的，由于 S_u 都是树上联通块，因次这等价于所有 S_u 的交非空，且这个交也一定是一个联通块。

P10064 [SNOI2024] 公交线路

显然只需要考虑叶子节点间的距离。对于点 u ，设新图上与它直接相连的点集为 S_u 。题目等价于任意两点 u, v 都满足 S_u, S_v 的交非空。

进一步的，由于 S_u 都是树上联通块，因次这等价于所有 S_u 的交非空，且这个交也一定是一个联通块。

一个合法方案对应一个联通块，那么考虑树上点减边容斥，统计每个点和每条边出现在联通块中的方案总数。

P10064 [SNOI2024] 公交线路

点和边是类似的，下面只考虑点的情况。直接计算全部相连比较困难，考虑容斥，钦定一些点不与枚举的根相连。

P10064 [SNOI2024] 公交线路

点和边是类似的，下面只考虑点的情况。直接计算全部相连比较困难，考虑容斥，钦定一些点不与枚举的根相连。

直接dp可以暴力合并根的所有儿子以及子树外的部分，但总复杂度 $O(n^3)$ 。

P10064 [SNOI2024] 公交线路

点和边是类似的，下面只考虑点的情况。直接计算全部相连比较困难，考虑容斥，钦定一些点不与枚举的根相连。

直接dp可以暴力合并根的所有儿子以及子树外的部分，但总复杂度 $O(n^3)$ 。

考虑到树上背包的复杂度是平方的，也就是说只合并儿子复杂度是可以接受的。那么对于子树外的部分，不再容斥而是钦定他们都必须与根相连，这样复杂度就优化到了树上背包的 $O(n^2)$ 。

CF1476F Lanterns

CF1476F Lanterns

CF1476F Lanterns

将判定转为最优化，设 f_i 表示前 i 个灯笼最多能覆盖多长的前缀。

CF1476F Lanterns

将判定转为最优化，设 f_i 表示前 i 个灯笼最多能覆盖多长的前缀。

转移分成 i 向左和向右两种情况。

CF1476F Lanterns

将判定转为最优化，设 f_i 表示前 i 个灯笼最多能覆盖多长的前缀。

转移分成 i 向左和向右两种情况。

对于 i 向右的情况是简单的，条件是 $f_i \geq i$ 。

CF1476F Lanterns

将判定转为最优化，设 f_i 表示前 i 个灯笼最多能覆盖多长的前缀。

转移分成 i 向左和向右两种情况。

对于 i 向右的情况是简单的，条件是 $f_i \geq i$ 。

对于 i 向左的情况，显然前面要存在一个 j 使得 $[1, f_j], [i - p_i, i - 1]$ 拼起来可以覆盖 $[1, i - 1]$ 。不存在则说明前面存在不能被前 i 个点覆盖的位置。而 $[j + 1, i - 1]$ 中的点都一定指向右侧。

CF1476F Lanterns

将判定转为最优化，设 f_i 表示前 i 个灯笼最多能覆盖多长的前缀。

转移分成 i 向左和向右两种情况。

对于 i 向右的情况是简单的，条件是 $f_i \geq i$ 。

对于 i 向左的情况，显然前面要存在一个 j 使得 $[1, f_j], [i - p_i, i - 1]$ 拼起来可以覆盖 $[1, i - 1]$ 。不存在则说明前面存在不能被前 i 个点覆盖的位置。而 $[j + 1, i - 1]$ 中的点都一定指向右侧。

那么找到 $f_j + 1 \geq i - p_i$ 的最小的 j ，并计算中间的点向右侧的最远距离。使用二分+st表就能轻松维护，复杂度 $O(n \log n)$ 。

决策单调性

决策单调性优化是凸优化中比较基础且常见的一种，应用非常广泛。

决策单调性

决策单调性优化是凸优化中比较基础且常见的一种，应用非常广泛。

决策单调性是说dp存在一组最优转移点满足其有单调性。

决策单调性

决策单调性优化是凸优化中比较基础且常见的一种，应用非常广泛。

决策单调性是说dp存在一组最优转移点满足其有单调性。

常见的处理方法有分治法，二分队列等，复杂度通常为 $O(n\log n)$ 。二分队列需要相对较强的单调性，分治法只依赖上述性质，但较难处理动态问题。

决策单调性

决策单调性优化是凸优化中比较基础且常见的一种，应用非常广泛。

决策单调性是说dp存在一组最优转移点满足其有单调性。

常见的处理方法有分治法，二分队列等，复杂度通常为 $O(n\log n)$ 。二分队列需要相对较强的单调性，分治法只依赖上述性质，但较难处理动态问题。

二分队列的具体做法为：仅考虑前 i 个点的转移，那么 i 会转移向一段后缀，二分位置即可。

决策单调性

决策单调性优化是凸优化中比较基础且常见的一种，应用非常广泛。

决策单调性是说dp存在一组最优转移点满足其有单调性。

常见的处理方法有分治法，二分队列等，复杂度通常为 $O(n \log n)$ 。二分队列需要相对较强的单调性，分治法只依赖上述性质，但较难处理动态问题。

二分队列的具体做法为：仅考虑前 i 个点的转移，那么 i 会转移向一段后缀，二分位置即可。

分治法的具体做法为：暴力找出 mid 的转移点，左部分只转移 $\leq mid$ 的，右部分同理。

qoj1355. Rhythm Game

qoj1355. Rhythm Game

qoj1355. Rhythm Game

容易想到朴素的dp，设 $f_{i,j}$ 表示前 i 个音符，错过了 j 个，最后一个音符没有被击中的情况下的最大得分。记 $w_{l,r}$ 为恰好从 l 连击到 r 的得分，那么转移式为：

$$f_{i,j} = \max\{f_{i-1,j-1}, \max_{k < i-1} (f_{k,j-1} + w_{k+1,i-1})\}$$

qoj1355. Rhythm Game

容易想到朴素的dp，设 $f_{i,j}$ 表示前 i 个音符，错过了 j 个，最后一个音符没有被击中的情况下的最大得分。记 $w_{l,r}$ 为恰好从 l 连击到 r 的得分，那么转移式为：

$$f_{i,j} = \max\{f_{i-1,j-1}, \max_{k < i-1} (f_{k,j-1} + w_{k+1,i-1})\}$$

直接计算复杂度 $O(n^3)$ ，考虑优化。

qoj1355. Rhythm Game

观察到题目中的贡献系数很特殊，考虑 $w_{l,r}$, $w_{l-1,r}$, $w_{l,r+1}$, $w_{l-1,r+1}$ 之间的关系：

$$\begin{aligned}(w_{l-1,r} + w_{l,r+1}) - (w_{l-1,r+1} + w_{l,r}) &= (w_{l-1,r} - w_{l-1,r+1}) + (w_{l,r+1} - w_{l,r}) \\ &= -(C_{r-l+3} \cdot A_{r+1}) + (C_{r-l+2} \cdot A_{r+1}) \\ &= A_{r+1} \times (C_{r-l+2} - C_{r-l+3})\end{aligned}$$

qoj1355. Rhythm Game

观察到题目中的贡献系数很特殊, 考虑 $w_{l,r}$, $w_{l-1,r}$, $w_{l,r+1}$, $w_{l-1,r+1}$ 之间的关系:

$$\begin{aligned}(w_{l-1,r} + w_{l,r+1}) - (w_{l-1,r+1} + w_{l,r}) &= (w_{l-1,r} - w_{l-1,r+1}) + (w_{l,r+1} - w_{l,r}) \\ &= -(C_{r-l+3} \cdot A_{r+1}) + (C_{r-l+2} \cdot A_{r+1}) \\ &= A_{r+1} \times (C_{r-l+2} - C_{r-l+3})\end{aligned}$$

题目保证了 $C_j \geq C_{j+1}$, $A_i \geq 0$, 因此:

$$w_{l-1,r} + w_{l,r+1} \geq w_{l-1,r+1} + w_{l,r}$$

qoj1355. Rhythm Game

进一步可以推得对于 $a < b < c < d$, 有:

$$w_{a,c} + w_{b,d} \geq w_{a,d} + w_{b,c}$$

qoj1355. Rhythm Game

进一步可以推得对于 $a < b < c < d$, 有:

$$w_{a,c} + w_{b,d} \geq w_{a,d} + w_{b,c}$$

这是标准的四边形不等式, 有较强的决策单调性。

qoj1355. Rhythm Game

进一步可以推得对于 $a < b < c < d$, 有:

$$w_{a,c} + w_{b,d} \geq w_{a,d} + w_{b,c}$$

这是标准的四边形不等式, 有较强的决策单调性。

朴素的dp交换 i, j 两维之后可以视作 f_{j-1} 向 f_j 的一组转移, 使用二分队列或分治法解决即可。

qoj1355. Rhythm Game

进一步可以推得对于 $a < b < c < d$, 有:

$$w_{a,c} + w_{b,d} \geq w_{a,d} + w_{b,c}$$

这是标准的四边形不等式, 有较强的决策单调性。

朴素的dp交换 i, j 两维之后可以视作 f_{j-1} 向 f_j 的一组转移, 使用二分队列或分治法解决即可。

单次转移 $O(n \log n)$, $w_{l,r}$ 可以以 $O(n^2)$ 的时间预处理, 总复杂度 $O(n^2 \log n)$ 。

CF1988F Heartbeat

CF1988F Heartbeat

CF1988F Heartbeat

以全局最大值为分界线，前面只有前缀最大值，后面只有后缀最大值，设 $f_{i,x,y}$ 表示长度为 i 的序列，有 x 个前缀最大值， y 个上升点的方案数，类似的设 $g_{i,x,y}$ 表示后缀部分的情况。

CF1988F Heartbeat

以全局最大值为分界线，前面只有前缀最大值，后面只有后缀最大值，设 $f_{i,x,y}$ 表示长度为 i 的序列，有 x 个前缀最大值， y 个上升点的方案数，类似的设 $g_{i,x,y}$ 表示后缀部分的情况。

考虑怎么求 $f_{i,x,y}$ ，考虑最小值的位置，如果放在开头，那么会多一个前缀最大值。如果插在原本的非上升点前面，那么会多一个上升点。 g 的转移同理，复杂度 $O(n^3)$ 。

CF1988F Heartbeat

考虑合并前后两部分，前缀/后缀最大值个数相互独立，直接算进贡献即可。暴力枚举左右的长度，上升点数量，可得 $O(n^4)$ 解法。

CF1988F Heartbeat

考虑合并前后两部分，前缀/后缀最大值个数相互独立，直接算进贡献即可。暴力枚举左右的长度，上升点数量，可得 $O(n^4)$ 解法。

注意到枚举的各个量之间相对比较独立，考虑分部转移降低复杂度。对上升点数量分部处理，借助辅助数组转移即可，复杂度 $O(n^3)$ 。

P11714 [清华集训 2014] 主旋律

P11714 [清华集训 2014] 主旋律

P11714 [清华集训 2014] 主旋律

DAG计数经典题。

P11714 [清华集训 2014] 主旋律

DAG计数经典题。

强联通难以直接刻画，考虑容斥，计算非强联通图的数量。

P11714 [清华集训 2014] 主旋律

DAG计数经典题。

强联通难以直接刻画，考虑容斥，计算非强联通图的数量。

先缩点，非强联通图缩点后DAG上会有多于1个点。考虑以DAG上的总点数为代表元，进行二项式容斥，相当于钦定一组强联通分量的划分，计算对应的方案数，再根据强联通分量数量的奇偶性赋权 $1/-1$ 。

P11714 [清华集训 2014] 主旋律

那么考虑dp，设 f_S 表示点集 S 中的点强联通的方案数。

P11714 [清华集训 2014] 主旋律

那么考虑dp，设 f_S 表示点集 S 中的点强联通的方案数。

转移考虑枚举没有入度的强联通分量的并。

P11714 [清华集训 2014] 主旋律

那么考虑dp, 设 f_S 表示点集 S 中的点强联通的方案数。

转移考虑枚举没有入度的强联通分量的并。

另设 g_S 表示 S 集合划分为若干独立的强联通分量的带权方案和, g 的转移考虑枚举其中一个强联通分量来计算。 f, g 相互转移就能求出具体的值了。

P11714 [清华集训 2014] 主旋律

那么考虑dp, 设 f_S 表示点集 S 中的点强联通的方案数。

转移考虑枚举没有入度的强联通分量的并。

另设 g_S 表示 S 集合划分为若干独立的强联通分量的带权方案和, g 的转移考虑枚举其中一个强联通分量来计算。 f, g 相互转移就能求出具体的值了。

实现时需注意转移顺序的一些细节, 总复杂度 $O(3^n)$ 。

P5642 人造情感 (emotion)

P5642 人造情感 (emotion)

P5642 人造情感 (emotion)

先考虑求出 $W(U)$ 。

P5642 人造情感 (emotion)

先考虑求出 $W(U)$ 。

考虑dp，记 S_x 分别表示 x 子树内的路径集合，设 $f_x = W(S_x)$ 。

P5642 人造情感 (emotion)

先考虑求出 $W(U)$ 。

考虑dp, 记 S_x 分别表示 x 子树内的路径集合, 设 $f_x = W(S_x)$ 。

先考虑 f 的转移, 分成 x 被覆盖和不被覆盖两种情况。

P5642 人造情感 (emotion)

先考虑求出 $W(U)$ 。

考虑dp, 记 S_x 分别表示 x 子树内的路径集合, 设 $f_x = W(S_x)$ 。

先考虑 f 的转移, 分成 x 被覆盖和不被覆盖两种情况。

不被覆盖的情况把所有儿子求和就行了。

P5642 人造情感 (emotion)

先考虑求出 $W(U)$ 。

考虑dp，记 S_x 分别表示 x 子树内的路径集合，设 $f_x = W(S_x)$ 。

先考虑 f 的转移，分成 x 被覆盖和不被覆盖两种情况。

不被覆盖的情况把所有儿子求和就行了。

被覆盖的情况，可以枚举覆盖 x 的路径，这条路径的 lca 一定是 x 。然后对两侧的链做链求和就能得到对应贡献。

P5642 人造情感 (emotion)

然后考虑求最终的答案。 $f(u, v)$ 对应的方案中一定选择了 (u, v) 这条路径，其余部分由两段链和一棵反子树构成。对这些部分的贡献求和就能得到答案。

P5642 人造情感 (emotion)

然后考虑求最终的答案。 $f(u, v)$ 对应的方案中一定选择了 (u, v) 这条路径，其余部分由两段链和一棵反子树构成。对这些部分的贡献求和就能得到答案。

考虑另设 g_x 为 x 子树外的最大路径和。

P5642 人造情感 (emotion)

然后考虑求最终的答案。 $f(u, v)$ 对应的方案中一定选择了 (u, v) 这条路径，其余部分由两段链和一棵反子树构成。对这些部分的贡献求和就能得到答案。

考虑另设 g_x 为 x 子树外的最大路径和。

转移类似 f ，但需要多考虑一种情况：路径经过 x 但 lca 不是 x 。这种情况 lca 一定是 x 的祖先且路径的其中一个端点在 $fa(x)$ 的其他儿子中，挂在线段树上查询即可。

P5642 人造情感 (emotion)

然后考虑求最终的答案。 $f(u, v)$ 对应的方案中一定选择了 (u, v) 这条路径，其余部分由两段链和一棵反子树构成。对这些部分的贡献求和就能得到答案。

考虑另设 g_x 为 x 子树外的最大路径和。

转移类似 f ，但需要多考虑一种情况：路径经过 x 但 lca 不是 x 。这种情况 lca 一定是 x 的祖先且路径的其中一个端点在 $fa(x)$ 的其他儿子中，挂在线段树上查询即可。

答案考虑每一个 $f(x), g(x)$ 的出现次数即可，复杂度 $O(n \log n)$ 。

END

谢谢大家！